MEMORAE / PROJECT BRIEFING

Your project, explained.

From a WhatsApp message to a reliable personal assistant

A plain-language presentation and reference guide
Architecture, database, completed work, examples and next steps

STATUS SNAPSHOT | 28 OCTOBER 2026

Local foundation through M1-08 | Code baseline: f55fcfa
The live assistant is not launched. Real model accuracy and reminder delivery remain unverified.

What you have today

A substantial local foundation, with important connections still switched off.

Built and tested locally

The project can receive and safely store message events, identify users, record consent and timezone, and organize work in order.

Saved, validated plans can create reminder records and list items. The database records what happened and prepares a reply.

There are model and WhatsApp adapters, spending controls, receipt processing, and an evaluation runner with 60 synthetic cases.

Still needed for a live assistant

The regular worker does not call a live AI model or send WhatsApp messages. Its optional fixture is a fixed clarification response, not intelligence.

Saved reminders do not ring or send notifications yet. Scheduling, templates, uncertain-send recovery and production operations remain ahead.

Provider responses were simulated in tests. Human evaluation-case review and measured live model quality are still pending.

Think of a workshop with a tested engine and wiring. It is not yet a vehicle ready for passengers.

Your reading route

Read in order, or click a section below. Every page is also a PDF bookmark.

What was built: milestones and design 04

Follow a message through the system 08

Safety, AI, spending and delivery 10

Understand the database 18

Find files and run a local demo 24

Evidence, next steps and launch gates 27

Glossary and source map 32

From an empty project to reliable work

M0 through M1-04: give every message a safe path through the system.

Milestone	What was created	Why it matters
M0 / Foundation	Repository, API, signed intake, inbox/outbox, Docker setup and design documents.	Messages are stored before we say they were received.
M1-01 / Identity	User mapping, consent, timezone, STOP and local chat simulator.	Know whose request it is and whether work may proceed.
M1-02 / Isolation	Separate database roles and forced row-level security.	Alice must not read or change Bob's records.
M1-03 / Work	Renewable leases, recovery, retries and per-user ordering.	Two workers must not perform the same job at once.
M1-04 / Planning	OpenAI adapter, strict validation and immutable saved plans.	Interpret first; execute a recorded decision separately.

The model connection exists in code, but remote planning remains disabled in the application worker.

From safe actions to measurable quality

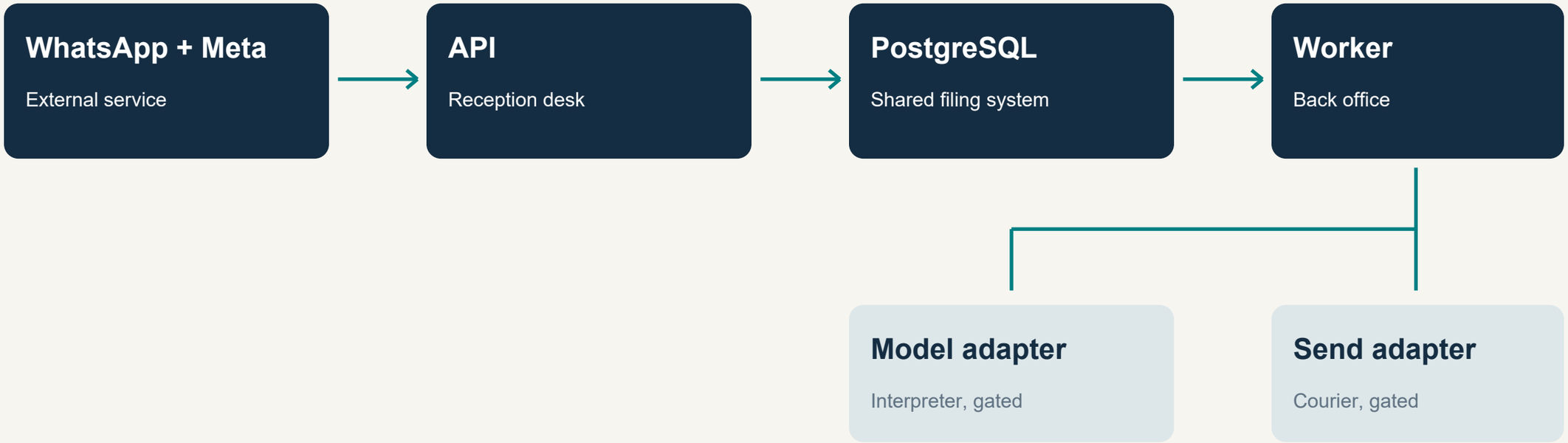
M1-05 through M1-08: control cost, record outcomes and make quality testable.

Milestone	What was created	Current boundary
M1-05 / Usage	Budget reservations, caps, rate snapshots, settlement and a kill switch.	Application spending starts paused. Live rates are unqualified.
M1-06 / Actions	Reminders, lists, ordered additions, action receipts and reply intents.	Local database effects work. Reminders do not fire.
M1-07 / WhatsApp	Text-send adapter, durable test dispatcher and receipt matching.	Receipt worker runs; live outbound sender is not wired in.
M1-08 / Evaluations	Real-provider runner, 60 agent-reviewed cases, scoring and versioned reports.	Mock run checked. Human review and paid measurement pending.

Local completion means implementation and tests exist. It does not mean launch approval or proven live delivery.

The system as a small office

One organized codebase, two main processes, one shared database.



The API and worker are two running programs from one codebase.
They coordinate through the database. External calls are kept outside short database transactions.

The external AI and outbound connections remain gated. Local intake, processing and receipt handling exist.

Why this design is deliberately simple

A modular monolith means one project with clear departments, not many separate services.

The tools and their jobs

TypeScript + Node.js: the language and runtime for the API and worker. Type checks catch mismatches before running.

Fastify: the API framework. **Zod:** a rule checker for incoming data and model decisions. **PostgreSQL:** durable storage.

Docker Compose: starts the local database in a repeatable container. A named volume preserves its data when the container stops.

The choices behind the design

An **API** is a reception desk: accept and file a request quickly. A **worker** does slower background tasks afterward.

One repository makes development and database transactions simpler for a small team. API and worker can still run separately.

Redis is optional and unused. Memory, calendar and dashboard folders are placeholders, not finished features.

A running Docker container is not a cloud deployment. This is currently a local development environment.

Example: add milk to grocery

After onboarding; real interpretation is currently tested with simulated model output.



You write: "Add milk to my grocery list." The API checks the signature, stores the event, and records a job. The worker checks consent and timezone.

The plan says: add_list_item, list_name = grocery, text = milk. The server chooses the owner; the model cannot choose another user.

The result: create or reuse your grocery list, append milk, save an action receipt and prepare a reply. All local effects commit together.

Retrying the same event reuses stable identifiers and receipts. It does not add a second milk entry from that same action.

Example: remind me tomorrow at 5pm

Dates need a reference time, a timezone, and an honest statement about delivery.



Fixed example: at 2pm on October 28, 2026 in America/Toronto, "tomorrow at 5pm" means October 29 at 17:00 with offset -04:00, or 21:00 UTC.

The plan stores that instant and the confirmed timezone. If execution happens after the due time, the whole plan is rejected and asks for a new date.

Today the reminder is only saved. Its reply explicitly says delivery is not enabled. M2 will create the scheduling and reminder-delivery workflow.

A reminder row is a saved request. It is not evidence that a notification was sent.

Permission before processing

Identity, consent and timezone are separate facts.

An onboarding conversation

1. Say **hello**. The local simulator displays a development notice and a unique consent command.

2. Copy **CONSENT [token]**. The token connects your answer to the notice you were shown.

3. Send **TIMEZONE America/Toronto**, check the preview, then copy **CONFIRM TIMEZONE [token]**.

4. A normal request becomes ready. **STOP** withdraws consent. **START** offers the notice again; it does not restore consent automatically.

What the system remembers

Identity is business phone plus sender ID. The same sender at another business number is a separate identity.

Consent records the notice version, time and audit evidence. Timezone confirmation has its own time and token. It is never guessed from a phone prefix.

A pending timezone change pauses eligibility. Old plans must match the current consent and timezone snapshots before acting.

The current notice is for synthetic local testing. Replace it before real users or live provider processing.

Alice and Bob share the building, not the files

Tenant means the person or account whose data a row belongs to.

Three database keys

Ingress login: a reception key for incoming events and jobs. It cannot browse reminder or list data.

Worker login: a trusted internal key for background processing across users. Never hand it to an end user.

Tenant login: a limited key with server-supplied user context. Row-level security allows only that user's permitted rows.

Two checks working together

Row-level security (RLS) is a cabinet rule: Alice sees only records labeled Alice. It is forced on protected tables.

Ownership-aware foreign keys check links too. Alice's item cannot point at Bob's list, even if someone guesses the ID.

Tests use real restricted database logins. Login/session authentication for a public dashboard is still future work; RLS does not replace it.

The server supplies identity. A model output or browser-provided user ID is not proof of permission.

What if the worker crashes?

Work is durable, and each worker temporarily borrows a job.

The checkout-slip analogy

A **lease** is a checkout slip with a worker name, random token and expiry. Only its current holder may commit a job's effects.

A heartbeat renews the slip while preparation runs. If the worker disappears, recovery makes the expired job available again.

A late worker with an old token is rejected. Database effects and final acknowledgement are checked and committed together.

Order and bounded retries

Requests for one person follow committed intake order. Different people can make progress independently.

Onboarding controls can pass an action backlog so STOP remains usable. Delivery-status messages have their own receipt stage.

Temporary failures retry with increasing delay.
Repeated/permanent failures become **dead work**, visible for repair rather than discarded.

Database arrival order is not perfect original sender order. Delayed provider messages cannot undo already committed effects.

The AI proposes; the application decides

A structured plan is an instruction form, not a free-form promise.

What goes to the model

The current message, confirmed timezone, reference time and processing time, plus fixed system instructions and the decision schema.

There is no conversation history or existing-list context. Unresolved references require clarification.

The decision is execute, clarify, noop or unsupported. Execute may contain one to five supported tool calls.

What happens after the answer

The application validates JSON, tool names, fields and future reminder times. Unknown fields and unsafe arguments are rejected.

It saves the validated plan with prompt hash, model identity, usage and provenance. Replays reuse that plan instead of asking again.

A schema-valid answer can still misunderstand intent. Evaluation and human review test the meaning, not just the format.

The worker defaults to model disabled. Fixture mode always returns a fixed clarification plan, not a real interpretation.

Reserve money before making a paid call

A reservation is like setting aside an envelope before ordering something.

An illustrative example

Imagine a $10 application cap and a $1 per-user cap for a UTC month. These are example limits, not provider prices or recommendations.

Before a model call, reserve its maximum estimated cost under both caps. Concurrent workers cannot each spend the same remaining balance.

When reliable usage returns, record the actual cost and release the unused hold. Integer microdollars avoid floating-point rounding: 1,000,000 equals $1.

When the outcome is uncertain

Versioned rate cards keep original prices attached to old reservations. A new rate does not rewrite history.

A timeout may still be billable. Unknown attempts keep their hold until evidence supports reconciliation.

The kill switch blocks new authorizations, not already-authorized calls. Actual overruns are recorded and pause further spending.

Local policy remains paused with zero caps. Standalone evaluation and future WhatsApp billing need separate cost controls.

All the changes, or none of them

A transaction is a sealed envelope of database changes.

Why the envelope matters

A plan can add milk and create a reminder. Its entities, action receipts, execution outcome, reply intent and completion form one transaction.

If reply storage fails or the lease expires before completion, the changes roll back. No hidden half-finished request is left behind.

An action receipt keeps the original result and entity ID. It proves a local change, not a WhatsApp delivery.

Details that avoid surprises

List names normalize Unicode, whitespace and case. Grocery and grocery share a list; Groceries stays separate.

Two intentional milk actions create two entries. Idempotency removes retries of the same action, not repeated user instructions.

Expired or unsupported digest reminders reject the whole plan. The scope is one-time reminders and additions, not recurrence or editing.

Idempotent means replaying the same work does not repeat its effect. It does not mean ignoring repeated requests.

Saved, accepted, delivered and read differ

Each word needs its own evidence.

A durable send attempt

The test dispatcher checks a phone/recipient allowlist, service window, identity snapshots and the global pause.

It records an attempt before calling the provider. A valid response records a provider ID and **accepted**, not delivered.

A timeout or failed result write leaves the attempt unknown or dispatching. It is not blindly resent.

Receipts arrive independently

Signed statuses may arrive early or out of order. The consumer stores the evidence before matching it.

Matching requires business phone, message ID and recipient. Old statuses cannot erase read/delivered evidence.

STOP suppresses queued replies and blocks later authorization. A send authorized before STOP can still finish; it cannot be recalled.

Receipt processing is active. The regular worker has no live sender; no WhatsApp message was sent during this work.

Test the model without giving it answers

The exam paper and answer key stay on different sides of the boundary.

The 60-case suite

10 reminders, 8 timezone cases, 10 lists, 8 ambiguous requests, 8 unsupported requests, 8 injection attempts, 4 no-ops and 4 multi-action cases.

48 development and 12 fixed holdout cases. Each has a Codex rationale; human review is pending.

The model sees only input and fixed time/zone context. It never gets expected answers, category, split or review notes.

Reports you can compare

Each run gets a unique folder: prompt/schema snapshots, dataset hash, model identity, Git revision, predictions, usage, latency and errors.

Scoring compares full actions and arguments. Equivalent timestamp offsets can match one instant. Errors and missing cases count as failures.

The live gate needs all 60 cases, 90% decision matches, and 100% on injection and scope. Mock reports cannot pass it.

The mock report proves the harness operates. It does not measure real model accuracy.

The database is a connected filing system

A table holds one kind of record; a row is one record.

users

Who owns the information?

action_plans

What should happen?

lists / reminders

What was created?

ingress_events

What arrived?

work_outbox

What still needs doing?

receipts / replies

What happened? What to say?

Conceptual map, not every foreign key. IDs connect rows; ownership constraints check that linked records belong to the same user.

A list item has its own ID, an owner user_id and a parent list_id. IDs connect records without copying everything.

Tables for intake, identity and work

Implemented names, with representative fields rather than every technical column.

Table	Plain-language job	Typical information
ingress_events	Incoming mail archive	Event, phone, sender, payload, sequence.
work_outbox	Job register	Event, stage/status, attempts, availability, lease token/expiry.
users	Person and permissions	Business phone + sender, consent, pause, timezone, service window.
identity_results	Onboarding decision	Event/user, disposition, stored reply, permission snapshots.
consent_events	Permission history	User/event, accepted or revoked, notice version/text, time.
user_preferences	User settings	User ID and reply language.
work_recovery_events	Repair history	Work ID, recovery/retry action, actor, time.

Raw events are internal transport records. A database table does not automatically expose a public endpoint.

Tables for plans and local actions

Connect an interpreted request to its actual effects.

Table	Plain-language job	Typical information
action_plans	Saved instruction form	Event/user, decision/version, model/prompt hashes, usage, snapshots.
lists	Named collection	ID, owner, display name, normalized name.
list_items	Entry inside a list	Owner/list, text, position, source event/action key.
reminders	Saved one-time reminder	Owner, title, due instant, timezone, pending status, source action.
action_receipts	Proof of each local action	Action key, plan/index/tool, entity, original result.
plan_executions	Overall plan outcome	Event/user/version, executed, blocked, expired or other outcome.
reply_intents	What to tell the user	Event/user, body, pending/suppressed, identity snapshots.

Action receipt: Was milk added? Reply intent: What should we tell the user?

Tables for model spending

Reservation, charge and operator history stay separate.

Table	Plain-language job	Typical information
usage_policy	Master spending settings	Enabled/paused, global cap, default user cap.
usage_user_limits	Individual exceptions	User ID and monthly cap.
model_rate_cards	Versioned price assumptions	Provider/model, rate version, token prices, input/output ceilings.
usage_global_periods	Global monthly totals	UTC period, reserved and spent.
usage_user_periods	User monthly totals	User + UTC period, reserved and spent.
usage_reservations	One attempted model operation	Work/lease, state, rate snapshot, held/settled usage.
usage_audit	Control change history	Actor, action, note and accounting/control evidence.

Application accounting does not replace provider invoices or guarantee a mispriced call cannot exceed a cap.

Delivery evidence and schema history

23 application tables in migrations, plus the migration runner's history table.

Table	Plain-language job	Typical information
whatsapp_attempts	Authorized send record	User/source, phone/recipient, attempt state, provider ID, delivery status.
whatsapp_receipts	Provider evidence	Ingress event, phone/message/recipient, status/time, matched attempt/user.
schema_migrations	Database upgrade checklist	Applied migration filename/checksum and application history. Created by the migration runner.

Unmatched receipts remain stored, without a tenant owner, until a safe match is found.

How the database grew, safely

A migration is a numbered instruction sheet that changes the structure.

Migration	What changed	Everyday explanation
0001	Ingress + work register	Build the mailroom and job book.
0002 / 0003	Identity, consent and notice binding	Add customer cards and versioned permission forms.
0004	Roles, forced RLS and ownership	Install locks and access rules.
0005 / 0006	Leases, recovery and persisted plans	Add checkout slips and instruction forms.
0007	Usage ledger and controls	Add spending envelopes and a budget office.
0008	Lists, reminders, actions and replies	Add service desks and their receipts.
0009	Send attempts and statuses	Add outbound dispatch and tracking.

Applied migrations are checksummed. Add a new file for a change; do not rewrite an applied migration.

Where things live in the repository

The GitHub repository is RappelAI; the project inside is called Memorae.

Location	What it contains	Use it to...
src/api + src/worker	The two program entry points	See what runs and in what order.
src/modules	Identity, WhatsApp, processing, lists, reminders, usage	Find one feature's business rules.
src/db + contracts + config	Database access, data shapes, settings	Understand boundaries and accepted inputs.
migrations + scripts	Schema history, setup, operator commands	Prepare and maintain the local system.
tests + evals	Automated checks and model benchmark	Find evidence that a change works.
docs + .github	Runbooks, roadmap, design, issue templates, CI	Plan, review and operate the project.
apps/dashboard + reserved modules	Future dashboard, memory, calendar and delivery boundaries	Plan later capabilities, not use finished features.

A folder organizes responsibility. It does not automatically mean a deployed feature exists.

Important files behind the examples

You do not need to read every line to follow the project.

From message to action

src/api/app.ts: HTTP and health routes.
src/modules/whatsapp/events.ts: normalize provider events.

src/db/ingress-store.ts: save events and jobs together.
identity/store.ts: onboarding changes.

processing/leases.ts + runner.ts: claim, renew, retry, complete.
plans.ts + execution.ts: save interpretations and apply actions.

From controls to evidence

modules/usage/service.ts: spending.
modules/whatsapp/dispatch.ts + receipts.ts: test sending and status evidence.

evals/run.ts + runner.ts: evaluation. **metrics.ts + reports.ts**: scoring and verification.

docs/backlog.md: ordered work. **docs/verification.md**: evidence and limits. **evals/README.md**: run the benchmark.

Short module paths above sit under src/modules/. Project root: C:/Users/neffa/Documents/ChatGPT/Memorae.

A demo you can follow today

Open PowerShell in C:/Users/neffa/Documents/ChatGPT/Memorae.

Try local onboarding

`npm run infra:up`
Start the database container.

`npm run onboarding:chat`
Follow hello, CONSENT, TIMEZONE and CONFIRM TIMEZONE.

Try a reminder request, then STOP and START. This demonstrates permission handling, not real reminder interpretation or scheduling.

`/quit` exits the simulator. Synthetic records remain in the local database.

Check the project

`npm run check`
Types, unit tests, dataset validation, build and formatting.

`npm run eval:run`
Generate a mock report without a provider call.

`npm run usage:control -- status`
Inspect controls. Do not casually resume spending.

Use docs/setup.md for API/worker startup. A fresh machine also needs npm ci, setup, migrations and role provisioning.

Do not delete Docker volumes unless you intend to erase local data. Never commit .env or expose API keys.

What the checks prove, and what they do not

Recorded evidence through M1-08; no new live-provider result is implied.

Evidence we have

49 unit tests passed: schemas, consent, time handling, adapters, scoring and failure behavior.

71 database tests/subtests passed at M1-07: real local PostgreSQL and restricted logins. M1-08 changed evaluation files and did not rerun that suite.

Type checks, build, formatting and 60-case validation passed. Migrations through 0009 were applied and safely rerun. API/worker startup passed at M1-07.

Evidence still needed

The CLI produced and verified a 60-row constant-noop mock report. Its live gate is false. It proves the harness, not model quality.

No real WhatsApp send, firing reminder, paid model evaluation, production load test or delivery-latency result has been demonstrated.

The source build is current. The original local Docker image has not been rebuilt for the later milestones.

A mock is a pretend external service with controlled answers. It tests failures without sending messages or paying for calls.

Three tracks to work on next

Keep one coding task active while external setup and review progress alongside it.

Track A: the next backend feature

M2-01: occurrence scheduling. Save a unique occurrence for each due reminder and an outbound intent in one transaction.

Test two schedulers, a crash before/after commit, an overdue reminder and changed eligibility. Two workers must not create two occurrences.

This turns a saved due time into durable delivery work. It still does not prove that a notification was delivered.

Tracks B and C: preparation

External setup: prepare Meta test-number access, a verified recipient and private credentials. Confirm current account rules and API support.

Model qualification: human-review the cases, select a model and authorize a bounded synthetic evaluation. Review failures before activating planning.

Product decisions: choose audience, languages, hosting region, retention and replacement consent before collecting real data.

Immediate actions: review the case expectations, prepare the test accounts, and make M2-01 the next coding issue.

The remaining delivery work, in order

M2 turns saved requests into a dependable reminder service.

Step	Build	Acceptance example
M2-01	Occurrence schema + due sweeper	Two sweepers save one occurrence and one send intent.
M2-02	Attempts, templates and status rules	Handle service-window boundaries and early receipts.
M2-03	Unknown-send recovery + bounded retry	A lost response does not trigger duplicate notification.
M2-04	Complete, cancel and snooze	An old queued intent cannot send a canceled reminder.
M2-05 / 06	Recurrence/DST + richer lists	Daily schedules survive clock changes; ambiguous edits clarify.
M2-07	Digest and quiet-hour preferences	Deferral is explicit, not a silently missed exact-time promise.

M1-07 supplies basic send/receipt pieces. M2 extends them into an operational reminder-delivery system.

Before a private beta

M3 is about operating the product responsibly, not just adding features.

Protect the experience

Add abuse/rate/plan limits and explain quotas. Do not accept promises that cannot be delivered under the product rules.

Implement verified export, deletion and retention. Deletion must cancel future work and handle stored data under the chosen policy.

Monitor queue age, failed/dead jobs, unknown sends, spend and delivery. Decide who responds when something breaks.

Prove the operations

Deploy staging API/worker and a managed database with private access, secrets, backups and tested restoration.

Run real-provider, failure and load tests. Check release migrations, pausing, recovery and truthful delivery status.

Start a small beta only after gates pass. Memory/search, voice, images, calendar, dashboard and billing follow the reliable core.

No calendar deadline is promised. External approvals, human review and measured results determine readiness.

A simple way to organize your work

Use the roadmap as the map and one issue board as the daily task list.

One task at a time

Columns: **Backlog, Ready, Doing, Review, Verify, Done**, plus Blocked. Keep one implementation item in Doing.

An issue states the problem, expected result, dependencies, a failure example and the evidence that will prove it works.

Example: "M2-01: two schedulers see the same due reminder; only one occurrence and outbound intent may be saved."

A repeatable routine

Choose a Ready task. Write acceptance examples first. Implement, test success and failure, then review the changes.

Commit and push the verified slice. Update the issue with evidence and the next action. Issue and pull-request templates are checked in.

A commit saves a checkpoint; push copies it to GitHub. Neither deploys the app nor backs up the live database.

The remote repository exists. No remote issue board was created by this work; use docs/backlog.md until you choose one.

A small glossary for reading the code

The same ideas appear repeatedly under technical names.

Term	Everyday meaning	Example here
API / webhook	Doorway / provider notification	Meta posts an event to the API.
Worker / queue	Background staff / job list	Claim a work_outbox row.
Transaction	All succeed or all cancel	Save item, receipt and reply together.
ID / foreign key	Record label / checked link	list_id points at the parent list.
Lease / idempotency	Temporary checkout / safe repetition	A retry reuses an action result.
Schema / migration	Record format / versioned change	0009 adds WhatsApp tracking tables.
Adapter / fixture	Translator / fixed test answer	OpenAI transport / constant clarification.
Hash / provenance	Fingerprint / where it came from	Exact prompt and dataset in a report.

A hash checks consistency, not proof of a provider call. A test is evidence for its tested scenario.

Saved checkpoints and source documents

Code baseline f55fcfa was pushed to main on October 28, 2026.

Checkpoint	Commit	Follow-along source
Setup + M1-01	36a88ac	docs/setup.md; docs/onboarding.md
M1-02 / M1-03	cf2d96d / 8d6938a	docs/tenant-isolation.md; docs/work-processing.md
M1-04 / M1-05	0729680 / 55a465f	docs/model-plans.md; docs/usage-controls.md
M1-06 / M1-07	3951624 / e138f83	docs/action-execution.md; docs/whatsapp-delivery.md
M1-08	f55fcfa	evals/README.md; evals/datasets/m1-v1-review.md
Design + ongoing plan	Repository documents	docs/architecture.md; docs/data-model.md; docs/roadmap.md; docs/backlog.md
Implementation authority	Code + migrations + tests	Source and migrations define actual behavior. Future design sections are not completed features.

Repository: github.com/AymenNF/RappelAI | Guide date: 2026-10-28 | No secrets or real-user examples included.